

Programmierung 2 - Sommersemester 2026

Prof. Dr. Markus Esch

Übung Nr. 9 Abgabe KW 27

1. Aufgabe

Schreiben Sie ein C-Programm, das vom Benutzer zunächst die Eingabe eines Geldbetrages (Nettopreis) erwartet, der mit `scanf()` eingelesen wird. Dann sollen 20% Mehrwertsteuer hinzugefügt werden (ergibt den Bruttopreis) und schließlich 2% Skonto abgezogen werden (ergibt den Rechnungsbetrag). Die Beträge sollen dann in folgendem Format ausgegeben werden:

```
Nettopreis           Euro xxxx.xx
+ 20% MwSt           Euro xxxx.xx
=====
Bruttopreis          Euro xxxx.xx
- 2% Skonto          Euro xxxx.xx
=====
Rechnungsbetrag      Euro xxxx.xx
```

2. Aufgabe

Implementieren Sie ein C-Programm, welches eine Zahl n vom Benutzer einliest und die n -te Fibonacci-Zahl sowohl iterativ als auch rekursiv berechnet und ausgibt.

Für beide Berechnungen soll die Zeit gemessen und ebenfalls ausgegeben werden.

Um die aktuelle Systemzeit in Millisekunden zu erhalten, können Sie in C die Funktionen `clock()` aus der Bibliothek `<time.h>` verwenden. Beispiel:

```
#include <time.h>

clock_t start, end;

start = clock();
// ... Code, dessen Laufzeit gemessen werden soll ...
end = clock();
```

3. Aufgabe

Schreiben Sie ein C-Programm, das die Collatz-Folge ¹ für eine gegebene Zahl erzeugt und die Anzahl der Schritte bis zum Erreichen von 1 berechnet.

1. Schreiben Sie eine Funktion `int collatzSteps(int n)`, die die Anzahl der Schritte berechnet.
2. Verwenden Sie die Collatz-Regeln:
 - Wenn die Zahl gerade ist, teile sie durch 2.
 - Wenn die Zahl ungerade ist, multipliziere sie mit 3 und addiere 1.
3. Verwenden Sie `scanf`, um die Startzahl vom Benutzer einzulesen.
4. Geben Sie die Collatz-Folge und die Anzahl der Schritte mit `printf` aus.

Beispiel:

Geben Sie eine Zahl ein: 6
 Collatz-Folge: 6 3 10 5 16 8 4 2 1
 Anzahl der Schritte: 8

¹<https://de.wikipedia.org/wiki/Collatz-Problem>

Lernziele:

Nach der Bearbeitung des Übungsblattes sollten Sie ...

- ... ein C-Programm mit Hilfe eines beliebigen Text-Editors schreiben und mit dem gcc-Compiler kompilieren und auszuführen können.
- ... in der Lage sein einfache C-Programme, unter Nutzung von Basisdatentypen, Operatoren, Kontrollstrukturen, scanf und printf zu implementieren..
- ... einfache Funktionen in C schreiben und aufrufen können, einschließlich der Übergabe von Parametern und der Rückgabe von Werten.
- ... die Verwendung von Schleifen (for, while, do-while) in C beherrschen.
- ... Eingabe- und Ausgabeoperationen in C durchführen können, einschließlich der Prüfung und Validierung von Benutzereingaben.

Zusatzaufgaben

Die folgenden Aufgaben sind Zusatzaufgaben und nicht Teil des Peer Reviews! Sie können die Aufgaben freiwillig bearbeiten, um die Inhalte weiter zu üben. Selbstverständlich können Sie auch Unterstützung und Beratung zu den Zusatzaufgaben erhalten.

1. Aufgabe

Schreiben Sie ein C-Programm, welches für die Typen `char`, `short`, `int`, `unsigned char`, `unsigned short` und `unsigned int` den maximalen und den minimalen Wert feststellt und ausgibt. Geben Sie zum Vergleich auch die Werte der Bibliothek `limits.h` der C Standard Library aus.

2. Aufgabe

Die Übergabe von Parametern an eine Funktion erfolgt in C nach dem call-by-value-Prinzip. Schreiben Sie ein Programm welches dies nachweist.

3. Aufgabe

Schreiben Sie ein C-Programm, das sowohl die Zahl Pi (π) mittels der Leibniz-Reihe als auch die eulersche Zahl (e) durch die Summation der unendlichen Reihe berechnet. Die Anzahl der Iterationen soll vom Nutzer eingegeben werden.

1. Schreiben Sie eine Funktion `double calculatePi(int terms)`, die π mit der angegebenen Anzahl von Termen berechnet.

2. Verwenden Sie die Formel der Leibniz-Reihe:

$$\pi = 4 \sum_{k=0}^n \frac{(-1)^k}{2k+1}$$

3. Schreiben Sie eine Funktion `double calculateE(int terms)`, die e mit der angegebenen Anzahl von Termen berechnet.

4. Verwenden Sie die Formel zur Berechnung von e :

$$e = \sum_{k=0}^{\infty} \frac{1}{k!}$$

5. Verwenden Sie `scanf`, um die Anzahl der Iterationen vom Benutzer einzulesen. Prüfen Sie die Nutzereingabe.

6. Geben Sie die Ergebnisse für π und e mit `printf` aus.

Beispiel:

Geben Sie die Anzahl der Iterationen ein: 100000

Die berechnete Zahl Pi mit 100000 Iterationen ist: 3.14159

Die berechnete Zahl e mit 100000 Iterationen ist: 2.71828

4. Aufgabe

Schreiben Sie ein C-Programm, welches zunächst eine Zahl n einliest und dann für n folgendes Berechnen:

- Die Fakultät von n . Implementieren Sie die Berechnung der Fakultät rekursiv.
- Die Summe der ungeraden Zahlen von 1 bis n .
- Die Summe der Quadratzahlen der Zahlen von 1 bis n .

Überprüfen Sie die Nutzereingabe und stellen Sie sicher, dass der Nutzer eine sinnvolle Eingabe tätigt.

5. Aufgabe

Schreiben Sie ein C-Programm, welches zwei Zahlen vom Nutzer einliest und den GGT der Zahlen berechnet. Berechnen Sie den GGT sowohl iterativ als auch rekursiv.

Messen Sie die Zeit für die rekursive und die iterative Implementierung und geben Sie diese ebenfalls aus.

6. Aufgabe

Schreiben Sie ein C-Programm, das eine Zahl vom Benutzer einliest und prüft, ob diese Zahl eine Primzahl ist. Die Prüfung, ob es sich um eine Primzahl handelt, soll in einer Funktion `isPrime` erfolgen.

7. Aufgabe

Schreiben Sie ein C-Programm, das vom Nutzer eine Angabe in Sekunden einliest und dies in Stunden, Minuten und Sekunden umrechnet. Z.B.:

```
> Geben Sie die Anzahl der Sekunden ein: 3661
> 3661 Sekunden sind 1 Stunden, 1 Minuten und 1 Sekunden.
```

8. Aufgabe

Schreiben Sie ein C-Programm, das als einfacher Taschenrechner funktioniert. Das Programm sollte zwei Zahlen einlesen und dem Nutzer die Möglichkeit geben, eine Rechenoperation (Addition, Subtraktion, Multiplikation, Division) auswählen. Das Ergebnis der Berechnung soll ausgegeben werden.